

HelperLearner - Setup, Installation & Deployment Guide

1. Local Development Setup

1.1 Prerequisites

- **Python:** 3.12+
- **PostgreSQL:** 14+ (or SQLite for quick testing)
- **Redis:** 6.0+ (optional, for Celery)
- **Git:** 2.x
- **Virtual Environment Manager:** venv (built-in) or pyenv

1.2 Clone Repository

Using HTTPS

```
git clone https://github.com/savinaysingh7/HelperLearner.git
cd HelperLearner
```

Or using SSH

```
git clone git@github.com:savinaysingh7/HelperLearner.git
cd HelperLearner
```

1.3 Setup Virtual Environment

Create virtual environment

```
python -m venv venv
```

Activate (Windows)

```
.\venv\Scripts\activate
```

Activate (Linux/Mac)

```
source venv/bin/activate
```

Verify activation (you should see (venv) in terminal)

1.4 Install Dependencies

Install all required packages

```
pip install -r requirements.txt
```

Verify installation

```
python -m django --version
```

Should output: 6.0.2

1.5 Environment Configuration

Create .env file in project root:

Copy example

```
cp .env.example .env
```

Edit with your settings
nano .env # or use any editor

Essential .env variables:

```
# Django
SECRET_KEY=your-super-secret-key-change-this-in-production
DEBUG=True
ALLOWED_HOSTS=localhost,127.0.0.1

# Database
DATABASE_URL=postgres://user:password@localhost:5432/helperlearner_dev

# Redis (optional, for cache/Celery)
REDIS_URL=redis://localhost:6379/0

# Razorpay (optional)
RAZORPAY_KEY_ID=rzp_test_xxxxx
RAZORPAY_KEY_SECRET=your-secret-key
RAZORPAY_WEBHOOK_SECRET=your-webhook-secret

# Gemini API (optional)
GEMINI_API_KEY=your-api-key

# Email Configuration
EMAIL_BACKEND=django.core.mail.backends.console.EmailBackend
EMAIL_HOST=smtp.gmail.com
EMAIL_PORT=587
EMAIL_HOST_USER=your-email@gmail.com
EMAIL_HOST_PASSWORD=your-app-password

# Sentry (optional)
SENTRY_DSN=https://xxx@sentry.io/xxx

# Site URL (for email links)
SITE_URL=http://localhost:8000
```

1.6 Database Setup

```
# Create database (PostgreSQL)
createdb helperlearner_dev

# Run migrations
python manage.py migrate

# Create superuser (admin account)
python manage.py createsuperuser
# Follow prompts: username, email, password
```

```
# (Optional) Load seed data
python manage.py populate_real_life_data
```

1.7 Run Development Server

```
# Start Django development server
python manage.py runserver

# Server running at http://localhost:8000
# Admin panel at http://localhost:8000/admin
```

1.8 Optional: Start Celery (Background Tasks)

```
# In a separate terminal, activate venv first

# Start Celery worker
celery -A helperlearner_root worker -l info

# Start Celery beat (scheduler)
celery -A helperlearner_root beat -l info
```

2. Docker Setup (Recommended)

2.1 Docker Compose

Prerequisites: - Docker Desktop (Windows/Mac) or Docker Engine (Linux) - Docker Compose 2.0+

2.2 Run Full Stack

```
# Build and start all services
docker-compose up --build

# Services running:
# - Django API: http://localhost:8000
# - PostgreSQL: localhost:5432
# - Redis: localhost:6379
# - Celery Worker: background
```

2.3 Docker Compose Commands

```
# Run in background
docker-compose up -d

# View logs
docker-compose logs -f web

# Run migrations
```

```
docker-compose exec web python manage.py migrate

# Create superuser
docker-compose exec web python manage.py createsuperuser

# Stop services
docker-compose down

# Remove volumes (reset data)
docker-compose down -v
```

2.4 dockerfile Overview

```
# Dockerfile
FROM python:3.12-slim

WORKDIR /app

# Install system dependencies
RUN apt-get update && apt-get install -y \
    postgresql-client libpq-dev gcc

# Copy requirements
COPY requirements.txt .

# Install Python dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY . .

# Collect static files
RUN python manage.py collectstatic --noinput

# Expose port
EXPOSE 8000

# Run Gunicorn
CMD ["gunicorn", "helperlearner_root.wsgi:application", "--bind", "0.0.0.0:8000"]
```

3. Database Initialization

3.1 PostgreSQL Setup (Local)

```
# Install PostgreSQL (macOS with Homebrew)
brew install postgresql
```

```
# Start PostgreSQL service
brew services start postgresql

# Create database
createdb helperlearner_dev

# Verify connection
psql -U postgres -d helperlearner_dev -c "SELECT 1;"
```

3.2 Django Migrations

```
# View pending migrations
python manage.py showmigrations

# Apply migrations
python manage.py migrate

# Create new migration after model changes
python manage.py makemigrations

# Dry-run migration (preview SQL)
python manage.py migrate --plan
```

3.3 Load Sample Data

```
# Load realistic test data
python manage.py populate_real_life_data

# This creates:
# - 50+ test users
# - 100+ help requests
# - 20+ freelance jobs
# - Test payments & escrow records
```

4. Testing

4.1 Run Tests Locally

```
# Run all tests
python manage.py test

# Run specific app tests
python manage.py test marketplace

# Run specific test class
python manage.py test marketplace.tests_features.HelpRequestModelTests

# Run with verbose output
```

```
python manage.py test --verbosity=2

# Run with coverage
coverage run --source='.' manage.py test
coverage report
coverage html # Opens in browser
```

4.2 Linting & Code Quality

```
# Format code with black
black .

# Check formatting (without changing)
black . --check

# Lint with flake8
flake8 .

# Type check with mypy
mypy .
```

5. Production Deployment

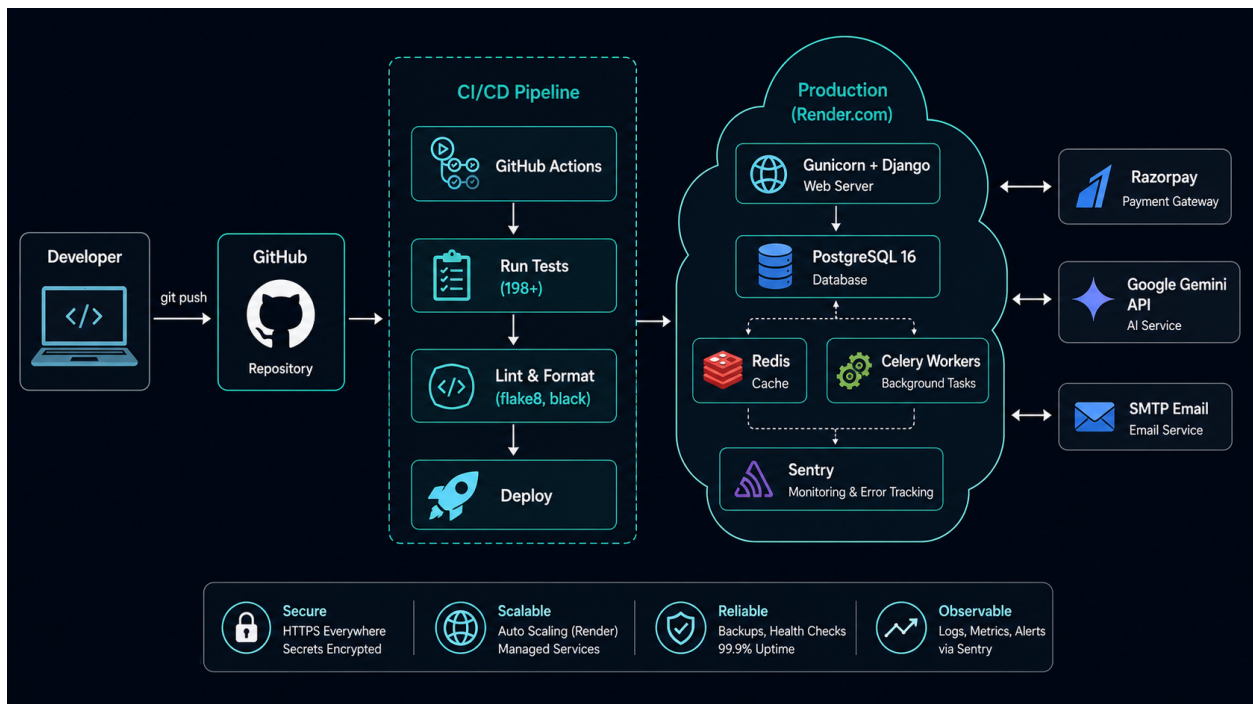


Figure 1: Deployment Architecture — CI/CD pipeline from GitHub to Render.com production

5.1 Render.com Deployment (Current)

Current Deployment Status: - Deployed on Render.com free tier - Auto-deploy from Different_High_Version branch - PostgreSQL provided by Render

Deploy Steps:

1. Push to GitHub:

```
git add .
git commit -m "Feature: new feature"
git push origin main
```

2. Render Auto-Deploy:

- Render watches the branch
- CI/CD pipeline triggers
- Tests run → deployment or rollback

5.2 Environment Variables (Production)

```
DEBUG=False
ALLOWED_HOSTS=helperlearner.onrender.com,www.helperlearner.onrender.com
SECRET_KEY=<use very strong key>
DATABASE_URL=<render-provided postgresql>
REDIS_URL=<render redis addon>
RAZORPAY_KEY_ID=<production key>
RAZORPAY_KEY_SECRET=<production secret>
GEMINI_API_KEY=<production key>
SENTRY_DSN=<sentry monitoring>
```

5.3 Database Backup (Production)

```
# Backup PostgreSQL
pg_dump -U user -h localhost -d helperlearner > backup.sql
```

```
# Restore
psql -U user -h localhost -d helperlearner < backup.sql
```

```
# On Render: Use Render dashboard → Backups
```

5.4 Health Checks

```
# Monitor application health
curl https://helperlearner.onrender.com/api/v1/health/
```

```
# Response:
# { "status": "healthy", "db": "connected", "redis": "connected" }
```

6. AWS Deployment (Alternative)

6.1 EC2 + RDS + ElastiCache

Architecture: - EC2: Run Django + Gunicorn + Nginx - RDS: PostgreSQL managed database - ElastiCache: Redis for cache/Celery - S3: Static files & media uploads

6.2 Deployment Script

```
#!/bin/bash
# deploy.sh

# SSH to EC2
ssh -i key.pem ubuntu@your-ec2-ip

# Clone repo
git clone https://github.com/savinaysingh7/HelperLearner.git
cd HelperLearner

# Setup Python environment
python3 -m venv venv
source venv/bin/activate

# Install dependencies
pip install -r requirements.txt

# Configure environment
cp .env.example .env
nano .env # Edit with RDS, ElastiCache details

# Run migrations
python manage.py migrate

# Collect static files
python manage.py collectstatic --noinput

# Start Gunicorn
gunicorn helperlearner_root.wsgi:application --bind 0.0.0.0:8000 --workers 4

# Setup Nginx (reverse proxy)
sudo apt install nginx
# Configure /etc/nginx/sites-available/helperlearner
sudo systemctl start nginx
```

7. Monitoring & Logging

7.1 Sentry (Error Tracking)

```
# helperlearner_root/settings/prod.py

import sentry_sdk
from sentry_sdk.integrations.django import DjangoIntegration

sentry_sdk.init(
    dsn=os.environ.get('SENTRY_DSN'),
    integrations=[DjangoIntegration()],
    traces_sample_rate=0.1, # 10% of transactions
)
```

Monitor at: sentry.io dashboard

7.2 Application Logs

```
# View Django logs
tail -f /var/log/helperlearner/django.log

# View Celery worker logs
tail -f /var/log/helperlearner/celery.log

# View Nginx logs
sudo tail -f /var/log/nginx/access.log
sudo tail -f /var/log/nginx/error.log
```

7.3 Performance Monitoring

```
# Track database query performance
from django.db import connection
from django.test.utils import CaptureQueriesContext

with CaptureQueriesContext(connection) as context:
    # Your code here
    pass

print(f"Queries executed: {len(context)}")
for query in context:
    print(f"Time: {query['time']}s SQL: {query['sql']}")
```

8. Scaling Considerations

8.1 Horizontal Scaling

Multiple Web Servers:

Load Balancer (HTTPS)

```
Web Server 1 (Gunicorn)
Web Server 2 (Gunicorn)
Web Server 3 (Gunicorn)
↓
Shared PostgreSQL
Shared Redis
Shared S3
```

8.2 Database Optimization

```
-- Add indexes for frequently queried columns
CREATE INDEX idx_user_trust_score ON accounts_customuser(trust_score DESC);
CREATE INDEX idx_request_status_date ON marketplace_helprequest(status, created_at DESC);

-- Connection pooling (PgBouncer)
# Install pgbouncer and configure connection pool
```

8.3 Cache Strategy

```
# Cache database queries
from django.views.decorators.cache import cache_page

@cache_page(60 * 15) # Cache for 15 minutes
def get_leaderboard(request):
    users = CustomUser.objects.order_by('-trust_score')[:100]
    return render(request, 'leaderboard.html', {'users': users})
```

9. Troubleshooting

9.1 Common Issues

Issue	Cause	Solution
ModuleNotFoundError	Missing dependency	<code>pip install -r requirements.txt</code>
psycopg2 error	PostgreSQL not installed	<code>pip install psycopg2-binary</code>
Connection refused	Redis not running	<code>redis-server</code> or <code>docker-compose up</code>
CSRF token missing	Cookie not sent	Check <code>CSRF_TRUSTED_ORIGINS</code> in settings
Static files 404	Not collected	<code>python manage.py collectstatic</code>

9.2 Debug Mode

```
# Enable detailed error pages
DEBUG = True

# Print SQL queries
from django.db import connection
print(connection.queries)
```

```
# Use Django debugger
import pdb; pdb.set_trace()
```

10. Maintenance

10.1 Dependency Updates

```
# Check for outdated packages
pip list --outdated

# Update specific package
pip install --upgrade django

# Update all
pip install --upgrade -r requirements.txt

# Generate new requirements file
pip freeze > requirements.txt
```

10.2 Database Cleanup

```
# Remove old expired help requests
python manage.py shell
>>> from marketplace.models import HelpRequest
>>> from django.utils import timezone
>>> from datetime import timedelta
>>> cutoff = timezone.now() - timedelta(days=30)
>>> HelpRequest.objects.filter(status='open', created_at__lt=cutoff).delete()
```

10.3 Performance Tuning

```
# Generate EXPLAIN ANALYZE reports
python manage.py shell
>>> from django.db import connection
>>> connection.queries_log # View slow queries

# Analyze query performance
EXPLAIN ANALYZE SELECT * FROM marketplace_helprequest WHERE status='open';
```

11. Checklists

Pre-Launch Checklist

- ☐ DEBUG = False in production
- ☐ SECRET_KEY is random & strong
- ☐ ALLOWED_HOSTS configured

- ☐ Database backed up
- ☐ HTTPS enabled
- ☐ Email configured
- ☐ Razorpay keys set
- ☐ Sentry monitoring active
- ☐ Rate limiting enabled
- ☐ CORS configured correctly
- ☐ Static files collected
- ☐ Celery workers running

Deployment Checklist

- ☐ All tests passing
 - ☐ Linting checks pass
 - ☐ Database migrations tested
 - ☐ Environment variables set
 - ☐ Backups created
 - ☐ Monitoring alerts configured
 - ☐ SSL certificate valid
 - ☐ Health checks passing
-

Summary

HelperLearner can be deployed: - Locally (Docker or native) - On Render.com (current - free tier) - On AWS (scalable) - On any Linux server (self-hosted)

Recommended for production: AWS with load balancing + RDS + ElastiCache

Current status: Running on Render.com with auto-deploy from GitHub